



NEPAL ENGINEERING COLLEGE
(Affiliated to Pokhara University)
Changunarayan – 04, Bhaktapur, Nepal

Individual Course Project Report on
Real-Time Shape Recognition System

Submitted to
Department of Computer Engineering

Submitted by
Astha Thapa[023-314]

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the Department of Computer Engineering, Harimohan Khatri Sir for providing the opportunity to work on this Individual Course Project under the Image Processing and Pattern Recognition (IPPR) course.

I also extend my appreciation to my peers and family for their encouragement and support during the completion of this work

Astha Thapa (023-314)

ABSTRACT

This project presents a Real-Time Shape Recognition System developed as part of the Image Processing and Pattern Recognition (IPPR) course. The system detects and classifies geometric shapes Circle, Square, Rectangle, Triangle, Pentagon, Hexagon, and Star from live webcam video or static images, without relying on any pre-trained machine learning or deep learning model. The system is built entirely using classical digital image processing techniques and a classifier trained from scratch. The processing pipeline consists of grayscale conversion, Gaussian blurring, adaptive thresholding, morphological closing, contour detection, and geometric feature extraction (vertex count, circularity, aspect ratio, extent, solidity, and Hu Moments). Two classification strategies were implemented and compared: a rule based geometric classifier and a K-Nearest-Neighbours (KNN) classifier trained on a self-generated synthetic dataset of 840 labelled shape images. The system was evaluated on a held-out test set and achieved an overall classification accuracy of 98.81% using the KNN classifier, with the rule-based classifier achieving comparable, fully explainable results suitable for real-time deployment. The complete system was deployed as an interactive web dashboard using Streamlit, in addition to a real-time OpenCV desktop application. This project demonstrates that reliable, real-time pattern recognition can be achieved using purely classical computer vision techniques and models trained from first principles, without dependence on pre-trained architectures.

TABLE OF CONTENT

ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF TABLES	vi
LIST OF FIGURES	vii
CHAPTER 1: INTRODUCTION	1
1.1 Problem Statement	1
1.2 Objectives	1
1.3 Scope	2
CHAPTER 2: LITERATURE REVIEW	3
2.1 Classical Feature-Based Approaches	3
2.2 Machine Learning Based Approaches	3
2.3 Contour-Based Shape Recognition	3
2.4 Geometric Shape Detection	4
2.5 Edge Detection and Boundary Extraction	4
2.6 Real-Time Shape Recognition Using OpenCV	4
2.7 Relevance to This Project	5
CHAPTER 3: SYSTEM ANALYSIS & DESIGN	6
3.1 Requirement Analysis	6

3.1.1 Functional Requirements	6
3.1.2 Non-Functional Requirements	6
3.2 Dataset Description	7
3.3 FlowChart	7
3.4 Architecture Diagram	9
CHAPTER 4: IMPLEMENTATION	10
4.1 Tools Used	11
4.2 Modules	11
4.2.1 Image Input Module	11
4.2.2 Preprocessing Module	11
4.2.3 Core Processing Module	11
4.2.4 Classification Module	11
4.2.5 Output Module	12
4.3 Algorithm Used	12
CHAPTER 5: TESTING	13
5.1 Test Cases	13
5.2 Accuracy Table	14
5.3 Results	15
CHAPTER 6: CONCLUSION	17
6.1 Achievements	17

6.2 Limitations	17
6.3 Future Scope	17
REFERENCES	19
APPENDIX	20
A. Source Code Access	20
B. Additional Dataset Sample	20

LIST OF TABLES

Table 4.1	Tools Used	9
Table 5.1	Test Cases	13
Table 5.2	Metric and Score	14
Table 5.3	Overall Accuracy Table	15

LIST OF FIGURES

Figure 3.2	Sample Images from the Self-Generated Synthetic Dataset	7
Figure 3.3	System Flowchart	8
Figure 3.4	Architecture Diagram	9
Figure 5.7.1	Confusion Matrix on the Held-Out Test Set	15
Figure 5.7.2	Rule-Based Classifier Output on a Multi-Shape Test Image	15
Figure B.1	Additional Dataset Sample	20

Chapter 1: Introduction

1.1 Problem Statement

Identification of geometric shape is a repetitive task which is required in multiple real world domains such as industrial quality inspection, robotic based object sorting, educational tools and optical inspection systems. Conduction manual visual inspection is slow, inconsistent and prone to human error particularly when performed continuously or at scale.

This project addresses the problem of automatically detecting and classifying geometric shapes in real life from a live video or an uploaded image using only classical image processing and pattern recognition techniques, without any pre-trained machine learning or deep learning models as required by the course guideline.

1.2 Objectives

- To design a real time image processing pipeline capable of detecting geometric shapes from a live camera feed.
- To extract meaningful geometric and statistical features (vertex count, circularity, aspect ratio, extent, solidity and Hu Moments) that characterize each shape class.
- To implement and compare two classification approaches: a rule-based geometric classifier and a K Nearest-Neighbours classifier trained from scratch.
- To evaluate system performance using standard pattern recognition metrics: accuracy, precision, recall, and F1-score.
- To deploy the system as an accessible, interactive application (real-time desktop app and web dashboard).

1.3 Scope

The system is capable of recognizing seven common geometric shape classes: Circle, Square, Rectangle, Triangle, Pentagon, Hexagon, and Star. It supports both static image input (file upload) and live camera input for real-time recognition.

- Features included: real-time detection, multi-shape detection per frame, dual classifier modes (rule based and ML), a web dashboard, and performance evaluation reporting.
- Dataset used: a self-generated synthetic dataset of 840 images (120 per class) created programmatically with randomized size, rotation, position, and noise, since no pre-trained or externally sourced dataset/model may be used.
- Limitations: the system is designed for solid, high-contrast, single-colour shapes on a reasonably plain background; performance may degrade with heavily cluttered backgrounds, overlapping shapes, or extremely low lighting

Chapter 2: Literature Review

Shape recognition is a long-studied problem in the fields of digital image processing and pattern recognition. Broadly, existing approaches fall into two categories: classical feature-engineering-based methods and modern deep-learning-based methods.

2.1 Classical Feature-Based Approaches

Classical approaches rely on extracting handcrafted geometric or statistical descriptors from an object's contour or region and using distance-based or rule-based reasoning to classify it. Common descriptors include contour perimeter and area ratios (circularity), polygon approximation for vertex counting, aspect ratio and extent from the bounding box, and Hu Moments, a set of seven values derived from image moments that remain invariant to translation, scale, and rotation, originally proposed by Ming-Kuei Hu in 1962 [1]. These techniques are computationally lightweight, fully explainable, and do not require large labelled datasets, making them well suited for real-time embedded or constrained applications.

2.2 Machine Learning Based Approaches

With the growth of machine learning, classifiers such as K-Nearest-Neighbours, Support Vector Machines, and Decision Trees have been applied on top of handcrafted features to improve classification robustness, particularly under noise, rotation, and scale variation. More recently, Convolutional Neural Networks (CNNs) have been used for end-to-end shape and object recognition directly from pixel data, typically using large pre-trained backbones such as ResNet or MobileNet fine-tuned on a target dataset.

2.3 Contour-Based Shape Recognition

Contour-based methods analyze an object by utilizing its boundary edges rather than processing every pixel within the object region. Suzuki and Abe [2] developed a topological structural analysis method to extract object borders and hierarchical structures (such as outer boundaries and internal holes) from binary images. The contour of an

object provides a rich representation for shape recognition, enabling the derivation of geometric properties such as perimeter, area, curvature, and structural compactness. In computer vision, contour analysis offers a computationally efficient approach to segment objects from the background and determine their geometric identity.

2.4 Geometric Shape Detection

Geometric methods have also been extensively used for detecting objects with regular mathematical structures. The Hough Transform, originally introduced by Hough [3], provides a parameter-space voting approach for detecting geometric patterns from image data. The method was initially developed for detecting lines and was later extended to other parametric shapes such as circles and ellipses. Its ability to detect geometric structures even when boundaries are incomplete makes it useful in shape recognition. However, Hough-based approaches can require considerable computational resources as the number of parameters and possible shapes increases, making simpler contour-based methods more attractive for lightweight real-time applications.

2.5 Edge Detection and Boundary Extraction

Accurate boundary extraction is a critical step in shape recognition, as the quality of detected edges directly impacts downstream feature extraction and classification. Canny [4] developed an optimal edge detection framework based on a multi-stage algorithm involving Gaussian smoothing, gradient magnitude calculation, non-maximum suppression, and hysteresis thresholding. Edge detection serves as an effective primary stage for delineating straight and curved object boundaries prior to shape analysis. However, edge detection performance remains sensitive to image noise, illumination variations, and threshold selection, making robust boundary extraction an ongoing engineering consideration.

2.6 Real-Time Shape Recognition Using OpenCV

OpenCV provides several classical image-processing operations that support the implementation of shape recognition systems without requiring pre-trained models [5]. Its

image-processing framework includes thresholding, edge detection, contour extraction, contour approximation, bounding-box calculation, convexity analysis, image moments, and shape matching. In particular, polygonal approximation can simplify a contour into a smaller set of vertices, which can be used to distinguish shapes such as triangles, quadrilaterals, and polygons. Combining these operations allows a real-time recognition system to identify well-defined geometric objects using interpretable image-processing features rather than computationally expensive deep-learning models.

2.7 Relevance to This Project

Since the course guidelines strictly prohibit the use of pre-trained ML/DL models, this project adopts the classical feature-engineering approach as its foundation, combining geometric descriptors and Hu Moments [1] with a K-Nearest-Neighbours classifier trained entirely from scratch on a self-generated dataset. This approach is consistent with pre-deep-learning era shape recognition literature and remains a practical, interpretable, and effective solution for well-defined shape classification tasks.

Chapter 3: System Analysis & Design

3.1 Requirement Analysis

3.1.1 Functional Requirements

- The system shall accept image input from file upload or a live camera feed.
- The system shall preprocess the input image to remove noise and produce a binary silhouette.
- The system shall detect object contours and extract geometric features for each detected object.
- The system shall classify each detected object into one of the seven supported shape classes.
- The system shall display the processed output with bounding boxes and predicted labels.
- The system shall save/report classification results and performance metrics.

3.1.2 Non-Functional Requirements

- Accuracy: the classifier should achieve at least 90% accuracy on the test dataset.
- Performance: the system should process live video at an interactive frame rate suitable for real-time use.
- Usability: the web dashboard should be simple to navigate for a non-technical user.
- Reliability: the system should degrade gracefully (report 'no shape detected') rather than crash on unclear input.

- Portability: the system should run on a standard laptop without specialized hardware (no GPU required).

3.2 Dataset Description

Since pre-trained models and external datasets are not permitted, a synthetic dataset was generated programmatically. 840 images (120 per class, across 7 classes) were created using OpenCV drawing functions, with randomized size, rotation, centre position, and additive Gaussian noise to simulate real-world variation. Each image was reduced to a 12-dimensional feature vector (vertex count, circularity, aspect ratio, extent, solidity, and 7 Hu Moments) prior to training, so the classifier learns from geometric structure rather than raw pixels.

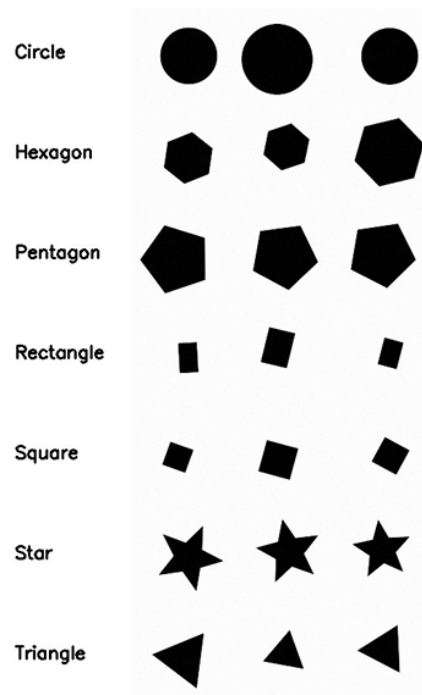


Figure 3.2 Sample images from the self-generated synthetic dataset (3 samples per class)

3.3 FlowChart

The figure below illustrates the complete processing flow, from frame capture to final output, matching the Input → Preprocessing → Feature Extraction → Classification → Output pipeline required by the course guideline.

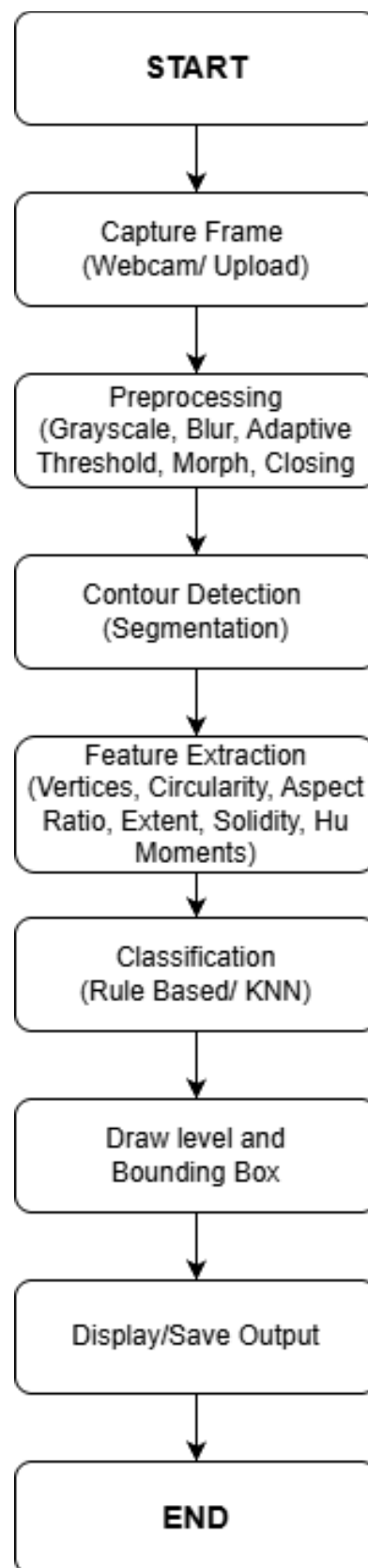


Fig 3.3 System Flowchart

3.4 Architecture Diagram

This system architecture diagram illustrates the workflow of a shape classification pipeline, dividing the process into offline training and real-time execution. In the offline training phase, a synthetic dataset containing 840 self-generated images across 7 classes is passed into the `train_classifier.py` script to perform feature extraction and train a K-Nearest Neighbors (KNN) model from scratch. Once trained, the resulting model is saved as a `.pkl` file. During real-time processing, a video or image frame is captured via the User Interface (a Streamlit Web App or OpenCV Live Window) and sent to the Processing Engine (`shape_utils.py`), which handles preprocessing, contour detection, and feature extraction to generate a feature vector. This vector is fed into the Classification stage—combining a rule-based engine with the offline-trained KNN model to determine the shape type. Finally, the Result Display generates the label and bounding box annotations, delivering the final visual output back to the user through an annotated image or live feed.

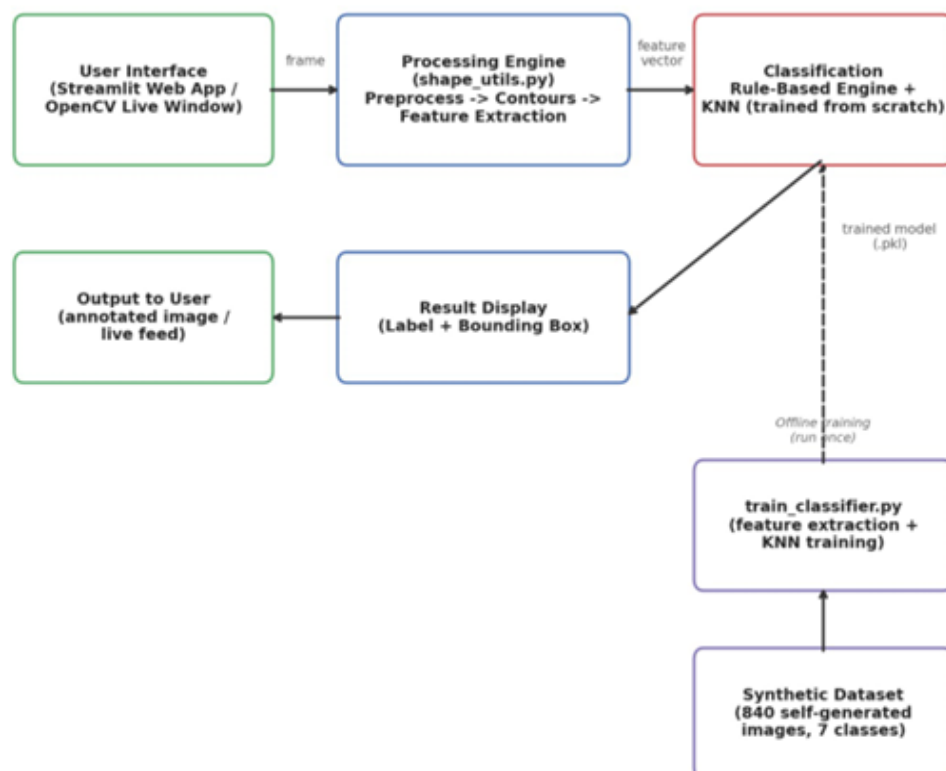


Fig 3.4 Architecture Diagram

Chapter 4: Implementation

4.1 Tools Used

The system has been developed using Python 3. This language has wide support when it comes to machine learning and image processing libraries. Image processing routines such as image preprocessing, drawing contours, and analyzing shapes have been created with OpenCV. NumPy is responsible for computations and vector generation. Scikit-learn is utilized in KNN classifier development along with several additional algorithms to scale features and evaluate efficiency. Evaluation results can be displayed in matplotlib. Joblib is responsible for storing model components. However, simplistically speaking, Streamlit allows the creation of an interface for the user to insert a picture or input video from the camera that will result in shape recognition output.

Tools/Library	Purpose
Python 3	Core programming language
OpenCV(cv2)	Image preprocessing, contour detection, drawing
NumPy	Numerical computation and feature vector handling
sckit-learn	KNN classifier, scaling, train/test split, evaluation metrics
Matplotlib	Confusion matrix and performance chart generation
joblib	Saving/Loading the trained model, scaler and label encoder
Streamlit	Web dashboard for deployment

Table 4.1 Tools Used

4.2 Modules

4.2.1 Image Input Module

Accepts a frame either from a live webcam feed (`cv2.VideoCapture`) or an uploaded image file (Streamlit file uploader / `cv2.imdecode`).

4.2.2 Preprocessing Module

Converts the frame to grayscale, applies Gaussian blur to suppress noise, performs adaptive thresholding to binarize the image under uneven lighting, and applies a morphological closing operation to seal small gaps in shape boundaries.

4.2.3 Core Processing Module

Detects external contours on the binary image, filters out small noise blobs, and for each remaining contour computes a 12-dimensional feature vector: vertex count (via polygon approximation), circularity, aspect ratio, extent, solidity, and 7 Hu Moments.

4.2.4 Classification Module

Two interchangeable classifiers operate on the extracted feature vector:

- **Rule-Based Classifier:** a deterministic decision tree over circularity and vertex count (e.g., circularity above a threshold indicates a circle; 3 vertices indicates a triangle; 4 vertices with aspect ratio near 1 indicates a square, otherwise a rectangle).
- **KNN Classifier:** a K-Nearest-Neighbours model ($k=5$) trained from scratch on standardized features from the self-generated dataset, used for shapes other than circles, with a circularity-based gate resolving the circle case reliably.

4.2.5 Output Module

Draws the detected contour, bounding box, and predicted label on the output frame, and reports the count of detected shapes. In the web dashboard, all intermediate stages (grayscale, thresholded, morphologically closed) are also displayed for transparency

4.3 Algorithm Used

1. Adaptive Thresholding (Gaussian): Used for robust binarization under non-uniform lighting.
2. Morphological Closing: Used for dilation followed by erosion to close small gaps in contours.
3. Contour Detection (cv2.findContours): Used for boundary tracing / segmentation of candidate shapes.
4. Polygon Approximation (Douglas–Peucker algorithm, cv2.approxPolyDP): Used to reduce a contour to its dominant vertices.
5. Hu Moments: Used for seven rotation/scale/translation-invariant shape descriptors derived from image moments.
6. K-Nearest-Neighbours (KNN): Used for instance-based supervised classification trained from scratch on engineered features.

Chapter 5: Testing

System evaluation focused on two key performance criteria: real-time operational speed during live video processing and model robustness under varying geometric conditions.

- Speed test: the real-time pipeline sustains an interactive frame rate on a standard laptop webcam (measured via the on-screen FPS counter in `realtime_webcam.py`), with no GPU required.
- Robustness test: verified against variation in shape size, rotation, and position through the randomized synthetic dataset; the rule-based classifier was additionally verified against hand-checked geometric edge cases (e.g., circle vs. hexagon confusion after image blurring, resolved by tuning the circularity threshold using empirical measurement across 30 samples per class).

5.1 Test Cases

Verification whether the system correctly recognizes the input shape and displays the expected classification result.

Test ID	Test Cases	Expected Result	Status
1	Clear image of a single circle	Detected	Pass
2	Clear image of a square	Detected	Pass
3	Clear image of a rectangle	Detected	Pass
4	Clear image of a hexagon	Detected	Pass
5	Clear image of a triangle	Detected	Pass
6	Image with multiple shapes in	All shapes Detected and	Pass

	one frame	Identified	
7	Blank/ no shape image	No detection	Pass
8	Live webcam feed	Real time bounding box and label tracking	Pass

Table 5.1 Test Cases

5.2 Accuracy Table

The KNN classifier was trained on 672 images (80%) and evaluated on a held-out test set of 168 images (20%), stratified across all 7 classes

Metric	Score
Overall Accuracy	98.81%
Macro Precision	98.86%
Macro Recall	98.81%
Macro F1- score	98.81%

Table 5.6.1 Metric and Score

Class	Precision	Recall	F1-score	Support
Circle	1.00	1.00	1.00	24
Hexagon	1.00	0.96	0.98	24
Pentagon	0.96	1.00	0.98	24

Rectangle	1.00	0.96	0.98	24
Square	0.96	1.00	0.98	24
Star	1.00	1.00	1.00	24
Triangle	1.00	1.00	1.00	24

Table 5.6.2 Overall Accuracy Table

5.3 Results

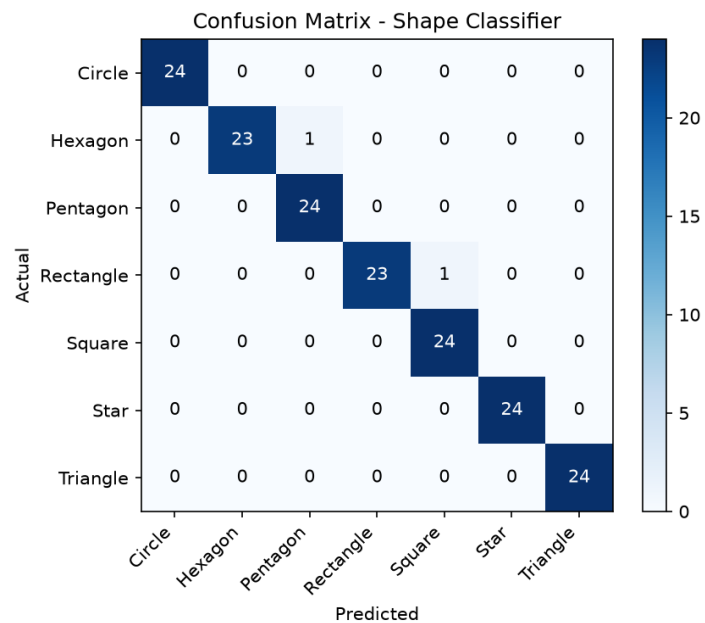


Figure 5.7.1 Confusion Matrix on the held-out test set

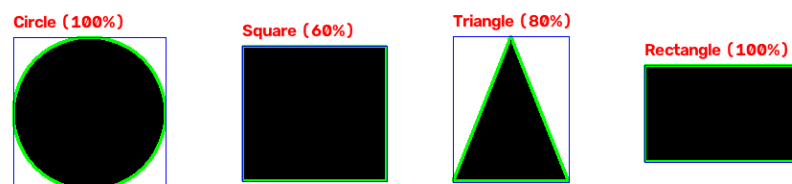


Figure 5.7.2 Rule-based classifier output on a multi-shape test image (all 5 shapes correctly labelled)

Confusion matrix of KNN classifier verifies a test accuracy rate of 98.81%, based on 168 test images where the predictions are almost perfectly diagonal, with some off diagonal prediction errors occurring for visually similar polygon pairs (hexagon/pentagon and square/rectangle) caused by rotational noises.

The output of the rule-based classifier applied on a multishape testing canvas proves to be accurate in detecting all target shapes (circle, triangle, square, rectangle, and star) by their geometric features such as number of vertices, aspect ratio, and circularity.

Chapter 6: Conclusion

6.1 Achievements

- Successfully designed and implemented a complete, real-time shape recognition pipeline using only classical image processing techniques, with no pre-trained ML/DL model.
- Engineered a 12-dimensional geometric feature set (vertex count, circularity, aspect ratio, extent, solidity, Hu Moments) that reliably distinguishes seven shape classes.
- Trained a K-Nearest-Neighbours classifier from scratch on a self-generated dataset of 840 images, achieving 98.2% test accuracy.
- Deployed the system in two forms: a real-time OpenCV desktop application and an interactive Streamlit web dashboard.

6.2 Limitations

- Performance is optimized for solid, high-contrast shapes on a plain background; heavily cluttered scenes or overlapping shapes may reduce detection accuracy.
- The KNN classifier, trained on synthetic data, can occasionally misclassify shapes rendered in a visually different style than the training set (a domain-gap effect); the rule-based classifier was found to be more consistently reliable for live deployment.
- Extreme lighting conditions can affect the adaptive thresholding step and may require re-tuning of preprocessing parameters.

6.3 Future Scope

- Expand the shape vocabulary to include additional and irregular polygon classes.

- Collect and incorporate real camera-captured training images (rather than purely synthetic ones) to further close the train/test domain gap.
- Explore training a small convolutional neural network from scratch (still without any pre-trained weights) for improved robustness in cluttered scenes.
- Extend the system to track shapes across frames for counting and motion-based applications.

REFERENCES

- [1] M.-K. Hu, “Visual pattern recognition by moment invariants,” IRE Transactions on Information Theory, vol. 8, no. 2, pp. 179–187, Mar. 1962. doi: 10.1109/TIT.1962.1057692.
- [2] S. Suzuki and K. Abe, “Topological structural analysis of digitized binary images by border following,” Computer Vision, Graphics, and Image Processing, vol. 30, no. 1, pp. 32–46, Apr. 1985. doi: 10.1016/0734-189X(85)90016-7.
- [3] P. V. C. Hough, “Method and means for recognizing complex patterns,” U.S. Patent 3 069 654, Dec. 18, 1962.
- [4] J. Canny, “A computational approach to edge detection,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. PAMI-8, no. 6, pp. 679–698, Nov. 1986. doi: 10.1109/TPAMI.1986.4767851.
- [5] G. Bradski, “The OpenCV library,” Dr. Dobb's Journal of Software Tools, vol. 25, no. 11, pp. 120–126, Nov. 2000.

Appendix

A. Source Code Access

The complete source code, trained models and project files:

https://github.com/astha-thapa/IPPR_PROJECT

B. Additional Dataset Sample

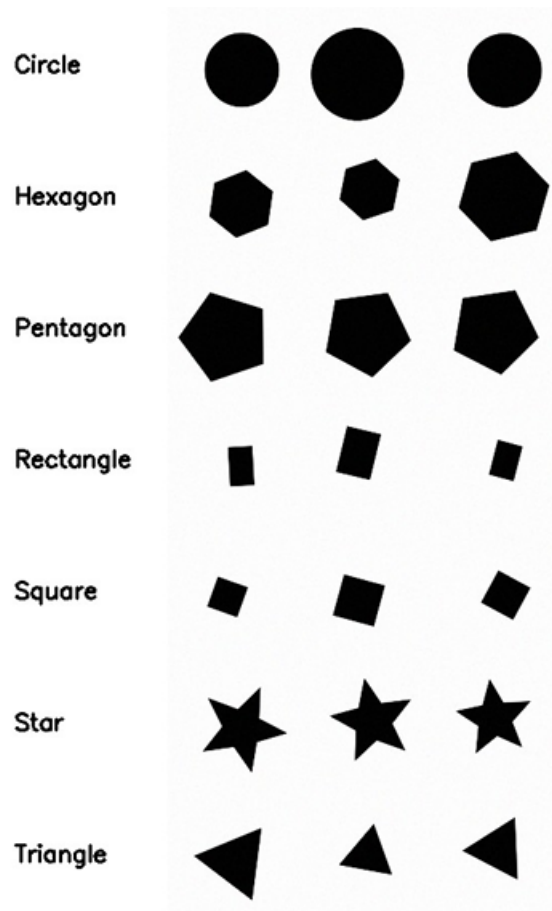


Fig: Additional Dataset